

Gráfico 1.

- Frecuencias con esquemas de saltos (2^k-1) y multiplicador $R=31$ ".

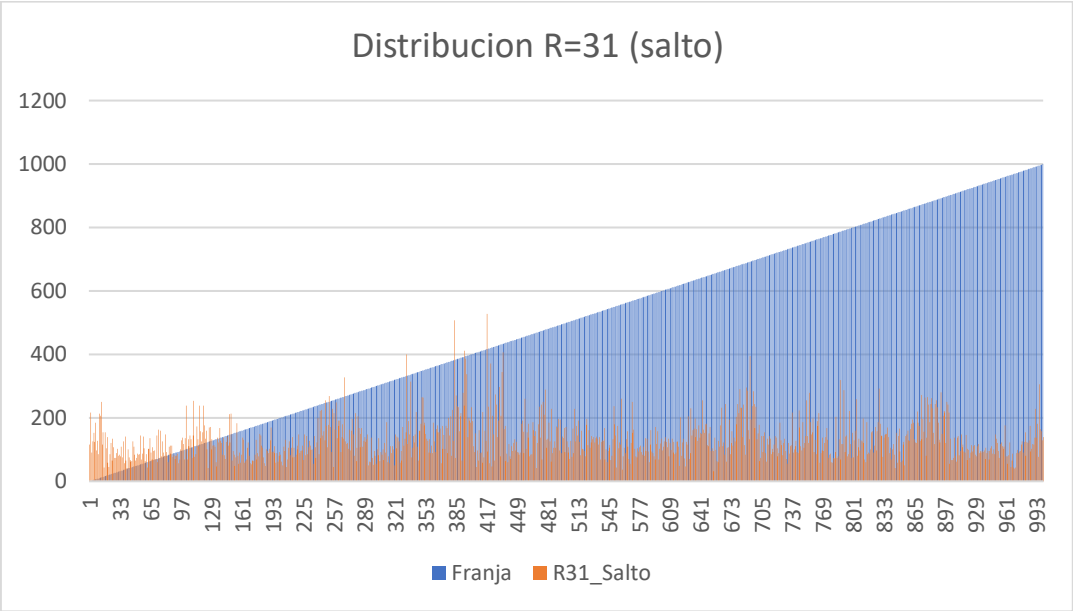


Gráfico 2.

- Frecuencias evaluando todos los caracteres y multiplicador $R=31$

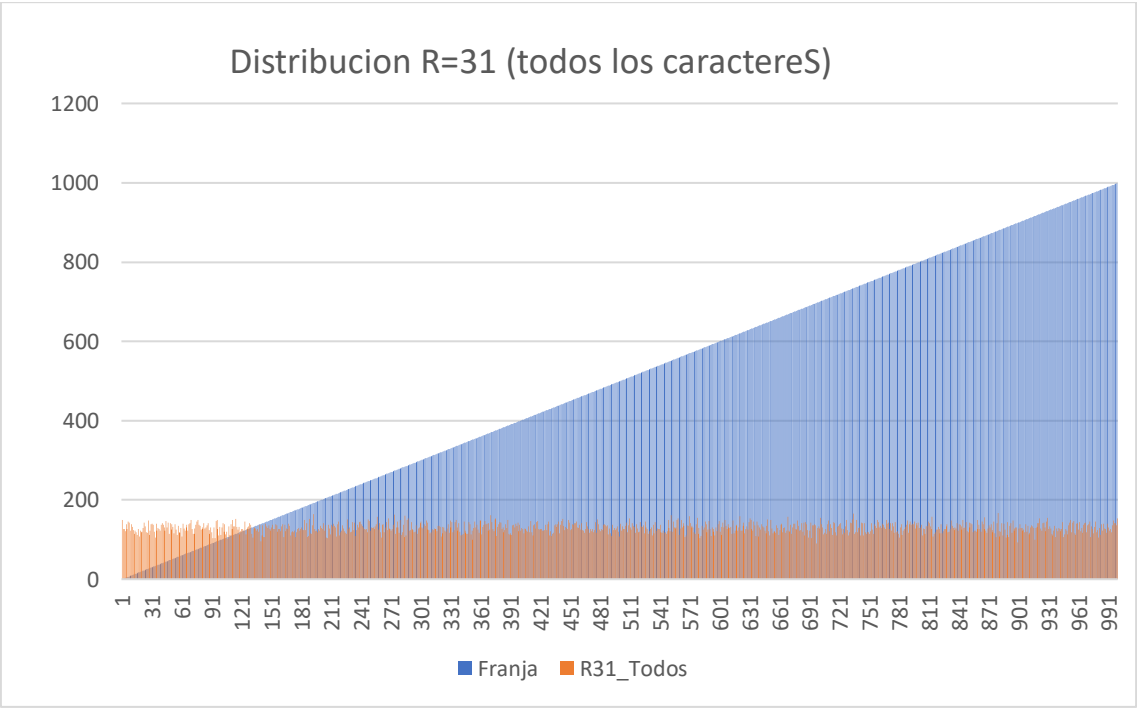
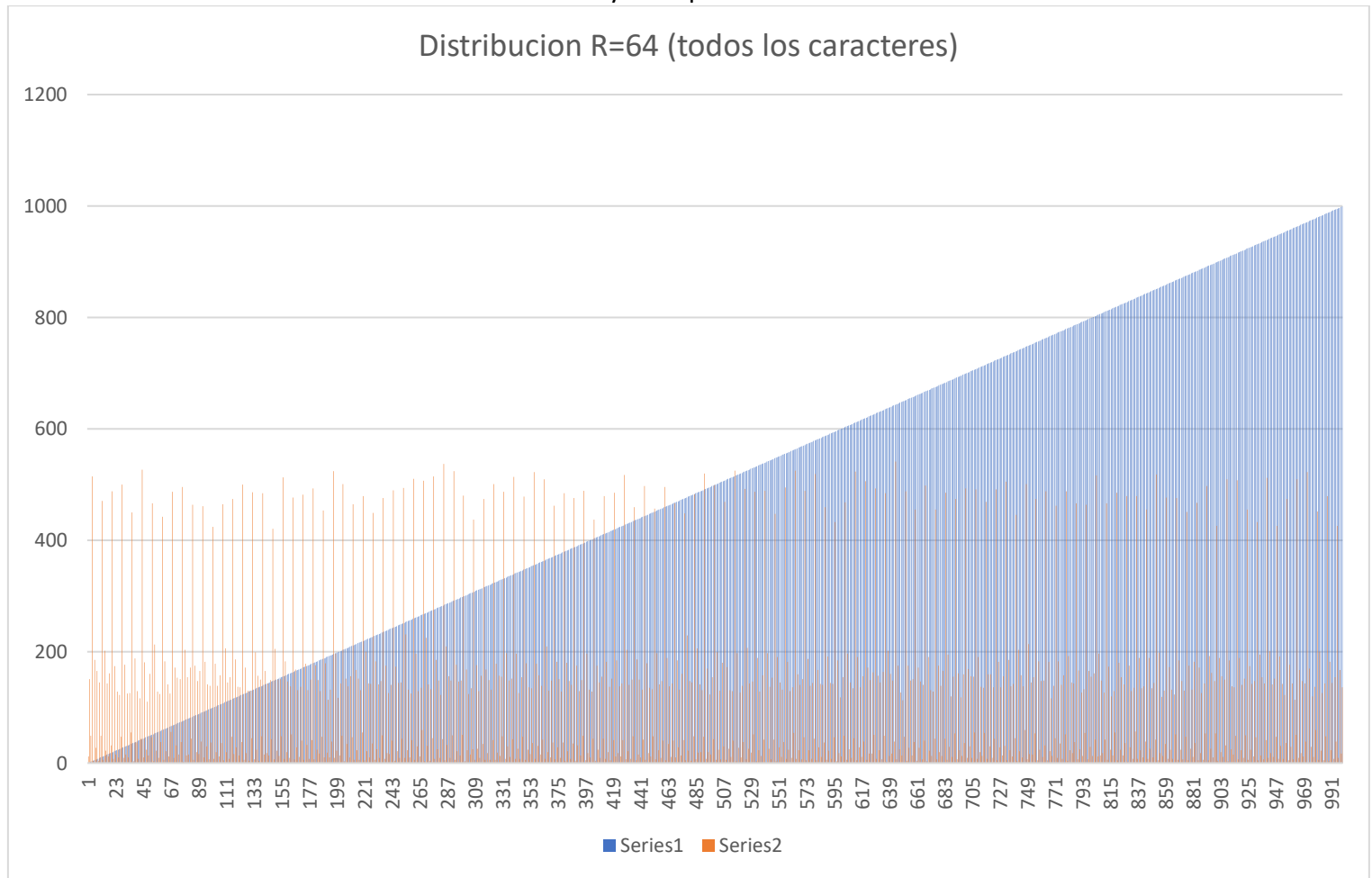


Gráfico 2.

- Frecuencias evaluando todos los caracteres y multiplicador R=64



Se comprueba visualmente que el esquema del código original (2^k-1), el cual va "saltando" caracteres de la cadena, es sumamente ineficiente. Al ignorar letras, genera colisiones masivas provocando que cientos de palabras terminen agrupadas en las primeras franjas (como se observa en el pico extremo de la Grafico 1), dejando el resto de la tabla hash casi vacía. En cambio, el esquema que evalúa **todos los caracteres** de la palabra (Grafico 2) distribuye las claves de manera mucho más equilibrada a lo largo de las 1000 franjas.

Tras procesar las claves, se constata que utilizar números primos como multiplicadores (R=31 o R=37) ayuda a distribuir uniformemente los valores hash en el módulo, reduciendo las colisiones. Por el contrario, utilizar potencias de 2 (R=32 o R=64) es una mala práctica para funciones hash en lenguaje natural, ya que al multiplicarse actúan como desplazamientos de bits (*bit-shifting*), perdiendo información de los primeros caracteres y generando agrupaciones indeseadas o "huecos" en la distribución de la tabla.